CS683 : Programming Assignment 2

Group Name: Cache Me If You Can

Group Members

Sai Thenge	24B1206
Khushi Kucheria	24B1219
Hamshini Kanuri	24B1253
Soham Dusane	24B3924

1 Task 2A: Instruction-Conscious L2 Replacement Policy (I-RRIP)

1.1 Results

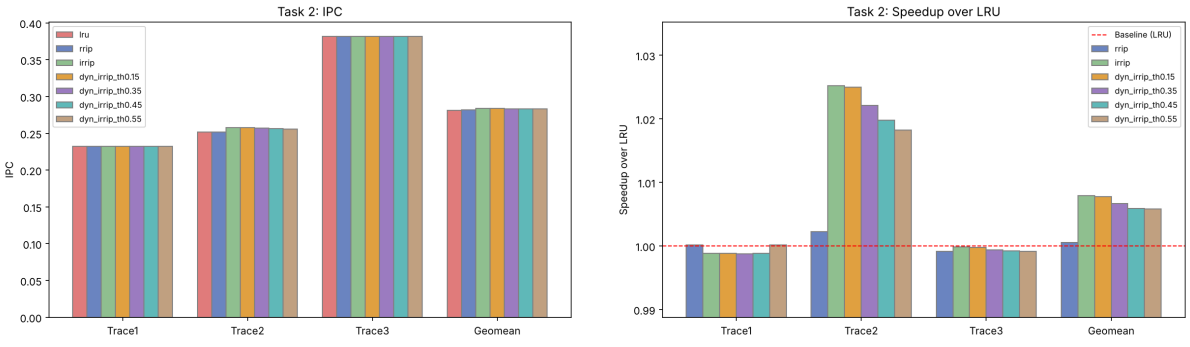
All runs use 25M warmup and 25M simulated instructions. Speedup is relative to the LRU baseline (no prefetching).

Table 1: IPC and speedup over LRU

	IPC			Speedup		
Trace	LRU	RRIP	I-RRIP	LRU	RRIP	I-RRIP
Trace1	0.2323	0.2323	0.2320	1.000	1.0002	0.9988
Trace2	0.2514	0.2520	0.2578	1.000	1.0023	**1.0252**
Trace3	0.3819	0.3816	0.3819	1.000	0.9991	0.9998
Geomean	0.2815	0.2816	0.2837	1.000	1.0005	**1.0079**

Table 2: L2 load MPKI (lower is better)

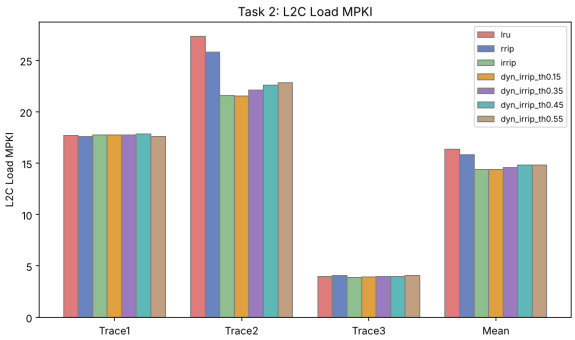
	L2C Load			L2 Instruction Load			L2 Data Load		
Trace	LRU	RRIP	I-RRIP	LRU	RRIP	I-RRIP	LRU	RRIP	I-RRIP
Trace1	17.68	17.60	17.76	10.35	10.54	**9.90**	7.33	7.06	7.86
Trace2	27.35	25.82	**21.56**	18.78	17.88	**12.43**	8.56	7.94	9.13
Trace3	3.96	4.04	**3.89**	1.13	1.20	**1.01**	2.83	2.84	2.89
Mean	16.33	15.82	**14.40**	10.09	9.87	**7.78**	6.24	5.95	6.63



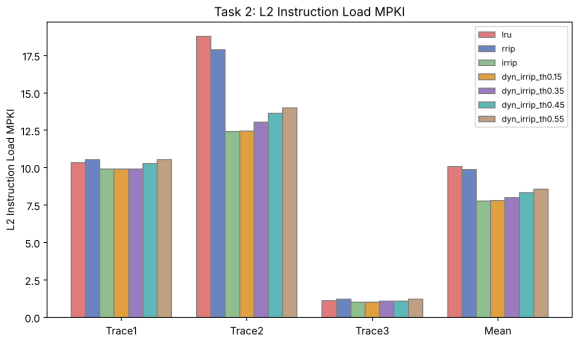
(a) IPC

(b) Speedup over LRU

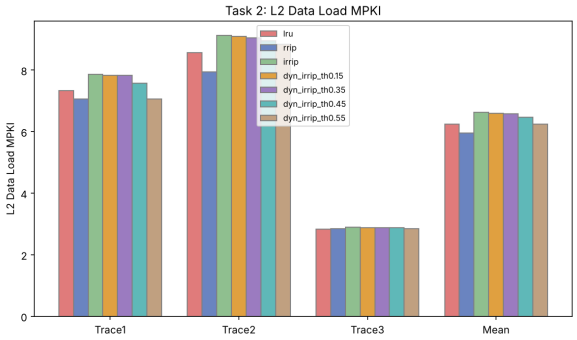
Figure 1: Task 2A performance



(a) L2C load MPKI



(b) L2 instruction load MPKI



(c) L2 data load MPKI

Figure 2: Task 2A L2 load MPKI

1.2 Design Rationale

Server-style workloads have instruction footprints larger than the L1I (Trace2 has an L1I MPKI of 40). An instruction miss that also misses in the L2 stalls the frontend and starves the whole pipeline, while data misses can partly overlap with other work in the out-of-order core. An L2 instruction hit is therefore worth more than an L2 data hit, so I-RRIP modifies all three parts of RRIP to favour instruction lines:

- **Insertion (modified).**
  - Instruction lines are inserted at RRPV 0 (most protected); data lines at RRPV 2, as in SRRIP.
  - Code is reused heavily (loops, repeated function calls), so a newly fetched instruction line is very likely to be used again. Inserting it at RRPV 0 protects it from being evicted on the first scan of the set.
  - This applies only while the set holds fewer than 6 instruction lines (the *instruction cap*). Beyond that, instruction lines are inserted like data.
- **Promotion (modified).**
  - On a hit, an instruction line is promoted to RRPV 0, but a data line only to RRPV 1.
  - Data reuse is still rewarded, but a data line can never become as protected as a reused instruction line. If it is not reused again, it ages out sooner.

- **Eviction (modified).**
  - When both a data line and an instruction line are at the maximum RRPV, the data line is evicted first, unless the set already holds more instruction lines than the cap.
  - Plain RRIP picks whichever line comes first in the set; this change uses that otherwise arbitrary choice to keep instructions in the cache.
- **Instruction cap.**
  - Without a limit, instruction lines could fill entire sets and drive data misses up sharply.
  - The cap gives instructions priority while always keeping enough ways for data.

1.3 Observations

- I-RRIP reduces L2 instruction load MPKI on every trace, by about 23% on average and 34% on Trace2 (18.78 $\rightarrow$ 12.43).
- Trace2 has the largest instruction footprint (L1I MPKI of 40) and benefits the most: its total L2C load MPKI drops from 27.35 to 21.56, giving a 2.5% speedup. The geomean speedup over all traces is 0.8%.
- The cost is a slight rise in L2 data load MPKI on all traces, since data lines are inserted and promoted to less protected positions.
- On Trace1 and Trace3, where instruction pressure is lower, the instruction gain and the data loss roughly cancel and IPC stays flat.
- Always prioritising instructions only pays off when instruction pressure is high. This motivates the Dynamic I-RRIP of Task 2B, which turns the instruction prioritisation on only when it helps.

2 Task 2B: Dynamic I-RRIP

2.1 Design

- **Set dueling.** 32 leader sets always run plain RRIP; all other (follower) sets run either I-RRIP or RRIP depending on the current decision.
  - Leader sets are spread evenly over the cache: with stride $s = \text{L2C_SET}/32$, set i is a leader if $(i \bmod s) = (\lfloor i/s \rfloor \bmod s)$. This gives one leader per group of s sets at a different offset in each group, so leaders are not all aligned to the same index bits.
  - 32 leaders is a small fraction of the L2 sets, so the leaders lose little performance by being forced to RRIP, yet they see enough accesses for a stable estimate.

- **Decision logic.** Only demand accesses (not writebacks) to leader sets are counted. Every epoch of 4096 such accesses:

$$\text{share} = \frac{\text{instr. accesses} + w \cdot \text{instr. misses}}{\text{total accesses} + w \cdot \text{total misses}}, \quad w = 1$$

  If share > threshold, followers use I-RRIP for the next epoch; otherwise they use RRIP.

- **History.** After each decision the counters are halved rather than cleared, so the estimate tracks phase changes but is not dominated by a single noisy epoch.

- **Why leaders run RRIP.** The leaders measure the workload's instruction share under a neutral policy. If they ran I-RRIP, the policy would itself change the miss mix it is measuring.

2.2 Threshold Sensitivity

Table 3: Speedup over LRU vs threshold

Threshold	Trace1	Trace2	Trace3	Geomean
0.15	0.9988	**1.0250**	0.9997	**1.0078**
0.25	0.9988	1.0250	0.9997	1.0078
0.35	0.9988	1.0221	0.9994	1.0067
0.45	0.9989	1.0198	0.9993	1.0059
0.55	**1.0002**	1.0182	0.9991	1.0058
Static I-RRIP	0.9988	1.0252	0.9998	1.0079
RRIP	1.0002	1.0023	0.9991	1.0005

Table 4: L2 load MPKI vs threshold (mean over traces in the last row of each block)

	L2C Load			L2 Instruction Load			L2 Data Load		
Threshold	T1	T2	T3	T1	T2	T3	T1	T2	T3
0.15	17.74	21.55	3.90	9.92	12.45	1.01	7.82	9.10	2.89
0.25	17.74	21.54	3.90	9.92	12.44	1.01	7.82	9.10	2.89
0.35	17.74	22.08	3.95	9.92	13.03	1.07	7.82	9.05	2.88
0.45	17.85	22.58	3.95	10.28	13.63	1.08	7.57	8.95	2.87
0.55	17.60	22.83	4.04	10.54	13.99	1.20	7.06	8.85	2.84

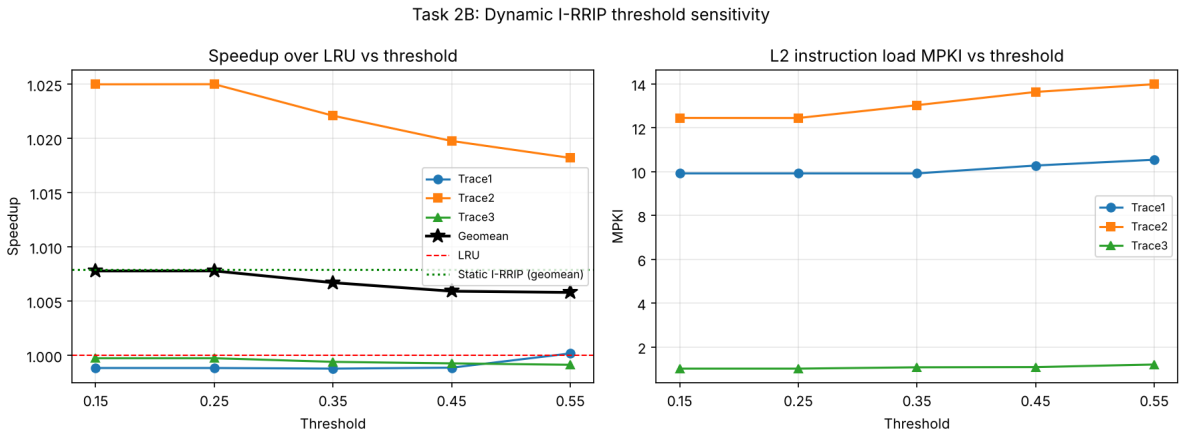


Figure 3: Task 2B: speedup and L2 instruction MPKI vs threshold

2.3 Observations

- **The threshold sets the instruction/data trade-off.** Raising it from 0.15 to 0.55 makes I-RRIP mode rarer: mean L2 instruction MPKI rises from 7.79 to 8.57 while data MPKI falls from 6.60 to 6.25.
- **Trace2 prefers a low threshold.** It is the most instruction-bound trace, so turning I-RRIP on more often helps it most (1.0250 at 0.15 vs 1.0182 at 0.55).
- **At 0.55, Trace1 and Trace3 never enter I-RRIP mode.** Their results are identical to RRIP down to the cycle count: the instruction share in their leader sets never exceeds 0.55.
- **Trace1 shows the benefit of switching off.** At 0.55 it recovers from 0.9988 (I-RRIP) to 1.0002 by avoiding the data-miss penalty.
- **Best threshold: 0.15** (geomean 1.0078). On these three traces Dynamic I-RRIP matches static I-RRIP but does not exceed it, because only Trace2 is strongly instruction-bound; the dynamic policy's protective effect (Trace1 at high thresholds) is too small a share of this trace set to pull ahead.